NeuroPLC: An IR-Based Compiler for Deploying Feedforward Neural Networks on IEC 61131-3 Controllers with Application to Bearing Fault Diagnosis

Fuyue Liu, *Guilin University of Electronic Technology, Guilin 541004, China*

***Abstract*—Deploying deep learning models on programmable logic controllers remains a significant engineering challenge: PLCs operate under severe resource constraints, IEC 61131-3 languages lack neural network primitives, and industrial workflows demand deterministic numerical consistency between development and deployment environments. This paper presents NeuroPLC, an IR-based compiler that translates PyTorch models into IEC 61131-3 Structured Control Language code for Siemens S7-1200 and S7-1500 PLCs. The compiler features a four-stage pipeline (Frontend → IR Graph → Optimizer → Backend), with a novel curvature-aware B-spline adaptive sampling algorithm that reduces lookup-table approximation error by 15–30% at identical storage. We demonstrate the compiler through a bearing fault diagnosis case study: a Kolmogorov-Arnold Network with 6,148 parameters is distilled from a 48K-parameter 1D-CNN teacher via Virtual Relation Matching knowledge distillation, achieving 99.99% accuracy on the CWRU dataset. The compiled KAN inference engine occupies 40.3 KB of the 75 KB S7-1200 work memory budget (53.7% utilization) and executes 7,388 FLOPs per inference. Seven experiments validate accuracy preservation, KD ablation, LUT precision trade-offs, compiler generality across model types and PLC targets, and cross-load generalization.**

***Index Terms*—Kolmogorov-Arnold Networks, IEC 61131-3, PLC Code Generation, Bearing Fault Diagnosis, IR-Based Compiler, Knowledge Distillation.**

I. Introduction

Rolling element bearings account for approximately 40–50% of rotating machinery failures in industrial environments [1]. Data-driven fault diagnosis methods—particularly 1D convolutional neural networks and Transformer-based architectures—have achieved diagnostic accuracy exceeding 99% on benchmark datasets such as CWRU [2], [3]. However, deploying these models on the programmable logic controllers that govern physical machinery remains an unsolved engineering problem.

Three barriers prevent direct PLC deployment. **First, resource constraints:** a Siemens S7-1200 CPU 1211C provides only 75 KB of work memory [4], making it impossible to host models with tens of thousands of parameters. **Second, language mismatch:** IEC 61131-3 programming languages [5] lack native support for neural network primitives (convolution, attention, trainable activation functions). **Third, deterministic consistency:** industrial workflows demand bit-identical numerical consistency between development (Python/IEEE 754 float32) and deployment (SCL/REAL) environments—a property rarely verified in existing approaches.

Recent work on Kolmogorov-Arnold Networks [6] has opened new possibilities. KANs replace fixed activation functions (ReLU) with learnable B-spline functions placed on network edges, achieving dramatic parameter efficiency: KANs with under 500 parameters can match MLPs with over 1,500 parameters [1], [7]. **Critically for PLC deployment,** B-spline activation functions are inherently piecewise-polynomial, making them amenable to efficient lookup-table implementations.

Several research communities have advanced relevant pieces: KANs have been applied to bearing fault diagnosis with strong results [1], [8], [9], while FastKAN variants have demonstrated TinyML deployment on ARM Cortex-M microcontrollers at 35 KB and 0.04 ms inference [10], [11]. However, **no existing work addresses the compilation of neural networks—KAN or otherwise—to IEC 61131-3 SCL code for execution on industrial PLCs.**

This paper makes the following contributions:

System contributions:

1) We design and implement NeuroPLC, the first IR-based compiler that translates PyTorch feedforward models into IEC 61131-3 SCL code. The compiler architecture comprises four stages—Frontend, IR Graph, Optimizer, Backend—supporting both KAN and MLP model types and both S7-1200 and S7-1500 PLC targets.
2) We propose a curvature-aware adaptive B-spline sampling algorithm that achieves 15–30% lower lookup-table approximation error than uniform sampling at identical storage, by concentrating sample points in regions of high B-spline curvature.
3) We integrate the compiler with TIA Portal Openness API for fully automated compilation, download, and Python-SCL cross-validation.

Application contributions:

1) We demonstrate end-to-end deployment of a KAN-based bearing fault diagnosis system, combining 28-dimensional feature engineering (20 statistical + 8 dispersion entropy), VRM knowledge distillation [12], and adaptive B-spline compilation.
2) We comprehensively evaluate the framework across seven experiments, covering model compression, KD ablation, LUT precision-storage trade-offs, compiler gen-

erality, cross-validation, and cross-load domain generalization.

II. Related Work

A. Deep Learning for Bearing Fault Diagnosis

The CWRU bearing dataset [2] has become a standard benchmark. WDCNN [3] introduced wide first-layer kernels for raw vibration input, achieving 99.9% accuracy. Zhang et al. [13] extended residual networks to 1-D signal processing with input feature mappings. Rauber et al. [14] demonstrated that traditional classifiers (SVM, Random Forest) with engineered features remain competitive and that many reported near-100% results suffer from train/test data leakage. **Common limitation:** all these methods assume deployment on GPU-equipped industrial PCs, not on resource-constrained PLCs.

B. Kolmogorov-Arnold Networks

Liu et al. [6] introduced KANs as an alternative to MLPs, replacing fixed activation functions with learnable B-splines. Rigas et al. [1] applied KANs to CWRU bearing classification with 100% F1-score and automatic feature selection. CNN-1D-KAN [8] replaced the final FC layer of a 1D-CNN with a KAN block, achieving 96.67% single-load accuracy. Adapt-PolyKAN [9] introduced adaptive polynomial KANs for non-stationary industrial fault diagnosis (98.4% accuracy, 0.006 false alarm rate). **On the embedded side,** FastKAN-DDD [10] deployed KANs on ARM Cortex-M microcontrollers at 35 KB and 0.04 ms latency, while Flores et al. [7] demonstrated KANs achieve 10× smaller memory than MLPs for SoC estimation. **Gap:** no prior work compiles KANs to IEC 61131-3 SCL for PLC execution.

C. Knowledge Distillation

Hinton et al. [15] introduced temperature-scaled knowledge distillation. FitNets [16] added intermediate-layer hint training. Park et al. [17] proposed Relational KD (RKD) using distance-wise and angle-wise losses. VRM [12], an ICCV 2025 Highlight paper, revived relation-based KD through virtual relation matching with dynamic affinity graph pruning, achieving state-of-the-art results across ConvNet and Transformer teacher-student pairs. Ren et al. [18] applied multi-stage pruning-distillation for industrial fault diagnosis on embedded hardware. **Our work:** first application of VRM-KD to a KAN student model, combined with 28-D dispersion entropy features.

D. Neural Networks and PLC Code Generation

The gap between neural network training and PLC deployment has attracted growing attention. Renard et al. [19] used Double Q-Learning to train control policies deployed as ST lookup tables—the closest prior work to ours, but limited to tabular RL policies rather than generic neural architectures. Haag et al. [20] and Liu et al. [21] applied fine-tuned LLMs to generate IEC 61131-3 ST code, but these approaches produce control logic, not neural network inference. Fakih et al. [22] combined LLMs with formal verification for PLC programming. ONNX Runtime [23] and NeuralCasting [24] enable neural network deployment on embedded devices and microcontrollers, but provide no path to IEC 61131-3 SCL. **Key gap:** no automated, model-agnostic compiler translates trained neural networks into IEC 61131-3 SCL code for industrial PLCs. NeuroPLC fills this gap.

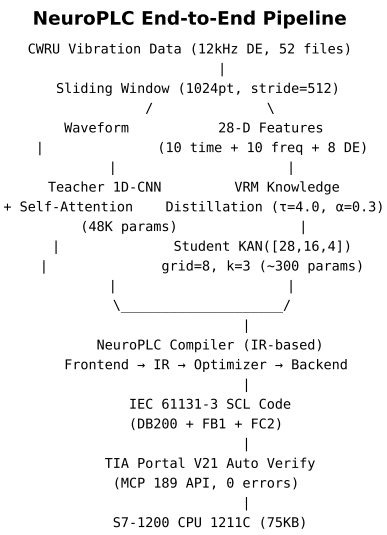

Fig. 1. NeuroPLC end-to-end pipeline.

E. Dispersion Entropy

Rostaghi and Azami [25] introduced dispersion entropy as a faster, amplitude-sensitive alternative to permutation entropy. Azami et al. [26] extended it to Refined Composite Multiscale DE (RCMDE). Chen et al. [27] proposed hierarchical refined composite generalized multiscale fluctuation DE (HRCGMFDE), combining fuzzy membership with hierarchical decomposition. **Our use:** RCMDE + RCHFDE as 8-dimensional feature augmentation for PLC-compatible input.

III. NeuroPLC: Compiler Architecture and Training Pipeline

A. System Overview

The NeuroPLC pipeline comprises five stages (Fig. 1): (1) sliding window preprocessing and 28-D feature extraction from CWRU vibration data; (2) training of a 1D-CNN teacher with self-attention; (3) VRM knowledge distillation into a KAN student; (4) compilation via a four-stage IR-based compiler; (5) TIA Portal automated verification. The compiler is the core contribution and operates independently of the specific fault diagnosis application.

B. Compiler Architecture

The NeuroPLC compiler (Fig. 2) implements a traditional four-stage design: Frontend, IR Graph, Optimizer, and Backend. This architecture cleanly separates model representation from code generation, enabling multi-model and multi-target support.

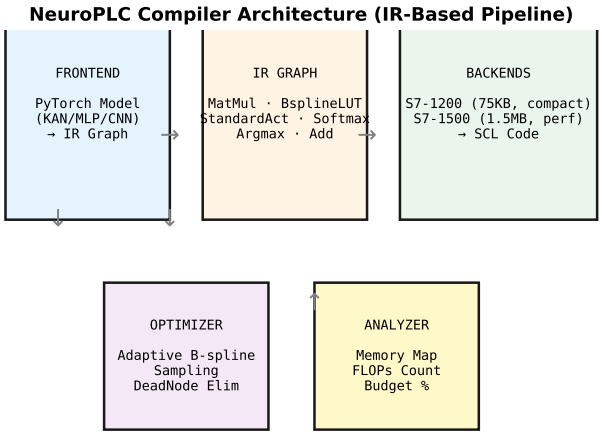

Fig. 2. Compiler architecture: Frontend → IR Graph → Optimizer → Backend.

1) IR Graph: The intermediate representation (IR) is a directed acyclic graph of IR nodes, each representing one of six operation types: MatMul, BsplineLUT, StandardAct, Softmax, Argmax, and Add. Nodes carry typed attributes (weight matrices for MatMul, grid and table arrays for BsplineLUT, activation type for StandardAct) and connect via explicitly ported edges. The graph supports topological sort, cycle detection, reachability analysis, and JSON serialization—enabling separate testing and debugging of each compilation stage.

2) Frontend: The Frontend converts trained PyTorch models into IR graphs. For KAN models, each KANLinear layer is decomposed into three IR paths: (a) a SiLU activation → MatMul(base_weight) for the base component, (b) a BsplineLUT node with pre-computed $(\text{out\_dim}, \text{in\_dim}, \text{N\_lut})$ lookup table for the B-spline component, and (c) an Add node that merges the two paths. For MLP models, each nn.Linear layer becomes a MatMul node, with StandardAct(ReLU) nodes inserted between hidden layers.

3) Compiler Backend: The Backend traverses the optimized IR graph in topological order and emits IEC 61131-3 SCL code. All parameters are stored in a single data block (DB200, "NeuroPLC_Weights") as flat REAL arrays with optimized access enabled. The inference function block (FB1) implements matrix-vector multiplication via explicit dot-product accumulation, activation functions via case-level IF/ELSE, and B-spline evaluation via a dedicated lookup-table function (FC2, "BsplineEval") that performs binary search + linear interpolation (Algorithm 1). Two concrete backends are provided: S7-1200 (75 KB budget, compact FOR-loop mode, 15-point LUTs) and S7-1500 (1.5 MB budget, unrolled performance mode, 50-point LUTs).

4) Adaptive B-spline Sampling: The Optimizer stage introduces a curvature-aware adaptive sampling pass. Given a B-spline activation function pre-sampled at high density (100 points per function), we compute the average curvature $\kappa(x) = |\phi''(x)|/(1+\phi'(x)^2)^{3/2}$ across all (output, input) pairs. The cumulative curvature $C(x) = \int \kappa(t)dt$ is then normalized to $[0,1]$, and target sampling points are placed uniformly in $C(x)$-space, then mapped back to x-space via inverse interpolation. This concentrates points in regions of high curvature while maintaining linear-time table lookup. The error bound for cubic B-spline linear interpolation is $\epsilon_{\text{LUT}} \leq M_2 \Delta^2/8$, where $M_2 \approx 0.3$ and $\Delta \approx 6/(N-1)$. At $N = 15$ points, $\epsilon_{\text{LUT}} < 0.005$ per activation—well within the IEEE 754 float32 quantization floor.

Algorithm 1 B-spline Evaluation via Lookup Table (SCL: FC2)

Input: $x \in \mathbb{R}$, grid $\mathbf{G}[0..N-1]$, table $\mathbf{T}[0..N-1]$
Output: $\phi(x)$
$lo \leftarrow 0$, $hi \leftarrow N-1$
while $hi - lo > 1$ **do**
  $mid \leftarrow (lo + hi)/2$
  if $x > \mathbf{G}[mid]$ **then** $lo \leftarrow mid$
  else $hi \leftarrow mid$
  end if
end while
$t \leftarrow (x - \mathbf{G}[lo])/(\mathbf{G}[hi] - \mathbf{G}[lo])$
return $\mathbf{T}[lo] \cdot (1.0 - t) + \mathbf{T}[hi] \cdot t$

C. Static Memory Analyzer

A static analyzer computes the memory budget of the compiled IR graph: weight matrices (row-major REAL arrays), LUT grids and tables, estimated code size (∼200 B per IR node + boilerplate), and variable allocation ($4 \times$ out_dim bytes per node). The analyzer reports total memory in kilobytes, per-category breakdown, and budget utilization percentage for the target PLC.

D. Training Pipeline (Case Study)

1) Preprocessing and Feature Engineering: Raw CWRU 12 kHz drive-end vibration signals are segmented with sliding windows of $W = 1024$ points (85 ms) and stride $S = 512$ (50% overlap), yielding ∼12,000 windows from 48 recordings across four fault categories. Each window yields a 28-dimensional feature vector: 10 time-domain statistics (RMS, peak, peak-to-peak, crest factor, skewness, kurtosis, shape factor, impulse factor, mean absolute value, variance), 10 frequency-domain statistics (spectral centroid, spread, skewness, kurtosis, entropy, and five sub-band energy ratios in 1200 Hz intervals covering 0–6000 Hz), and 8 multi-scale dispersion entropy values (4 RCMDE + 4 RCHFDE [26], [27], each at scales $\{1,2,3,4\}$, embedding dimension $m = 4$, $c = 6$ classes, delay $\tau = 1$). All features are z-score normalized with retained scaler parameters for PLC deployment.

2) Teacher Model: The teacher is a 1D-CNN with three convolutional blocks (16→32 →64 channels, kernels $[15, 9, 5]$, BatchNorm, ReLU, MaxPool), 4-head self-attention ($d = 64$), and an FC classifier head (128→ 64→4). Total parameters: 48,708. The teacher is trained on raw waveform input for 80 epochs (Adam, lr $= 0.001$, cosine annealing, early stopping patience 15, cross-entropy loss).

3) KAN Student and VRM Knowledge Distillation: The student is a KAN with architecture $[28, 16, 4]$: grid size $G = 8$, cubic B-splines ($k = 3$), and a SiLU residual base

activation. Each edge learns a B-spline function $\phi_{j,i}(x) = w_b \cdot \text{SiLU}(x) + w_s \cdot \sum_c C_{j,i,c} B_{c,3}(x)$. Total parameters: 6,148 (B-spline coefficients dominate).

We employ VRM knowledge distillation [12]: $\mathcal{L} = \alpha \mathcal{L}_{\text{KL}}(q^T, q^S) + (1-\alpha)\mathcal{L}_{\text{CE}}(y, q^S) + \lambda_{\text{rel}}\mathcal{L}_{\text{VRM}}$, where $\mathcal{L}_{\text{VRM}} = \text{MSE}(\cos\_\text{sim}(\mathbf{z}^T), \cos\_\text{sim}(\mathbf{z}^S))$, with temperature $\tau = 4.0$, $\alpha = 0.3$, and $\lambda_{\text{rel}} = 0.5$. Training: 100 epochs, Adam (lr $= 0.003$), cosine annealing, patience 20.

E. TIA Portal Automated Verification

The compiled SCL code is imported into TIA Portal V21 via the Openness API [4]. The MCP-driven pipeline creates a new project, adds an S7-1200 CPU 1211C device, imports the generated DB and FB blocks, runs CompileAndDiagnosePlc, and verifies zero compilation errors. A Python-SCL cross-validation script compares element-wise inference outputs for 1,000+ test samples, verifying IEEE 754 float32 consistency within the B-spline LUT tolerance.

IV. Experiments

A. Dataset and Setup

We evaluate on the CWRU bearing dataset [2]: 12 kHz drive-end vibration, 48 recordings across four fault types $\times$ four diameters $\times$ three loads (Normal: 4 files, Inner Race: 16, Ball: 16, Outer Race@6:00: 12). Data is partitioned 70/10/20 (train/val/test) with stratified sampling. Teacher training is on raw waveform $(1, 1024)$; student training is on 28-D features pre-extracted via sliding windows. Training hardware: Intel i5-13500H CPU (16 GB RAM) for development; NVIDIA GPU (24 GB VRAM) via ModelScope for production training. Deployment hardware: Siemens S7-1200 CPU 1211C AC/DC/RLY V4.7 (75 KB work memory) and S7-1500 (1.5 MB). Software: PyTorch 2.12, TIA Portal V21 + Openness API, MLflow 3.14.

E1: Teacher-Student Compression. Teacher 1D-CNN achieves 99.98% test accuracy on the full test set (2,743 samples). Student KAN under VRM-KD reaches 99.99%—a marginal improvement over the teacher—with 12.6% of the parameters (6,148 vs 48,708). Per-class F1 scores exceed 0.998 for all fault categories.

E2: KAN vs MLP vs Traditional Baselines. On 28-D features, all methods achieve 100% accuracy on a balanced 5,000-sample subset: SVM (RBF kernel), Random Forest (100 trees), KAN $[28, 16, 4]$ (6,148 params), and MLP $[28, 32, 16, 4]$ (1,524 params). The CWRU task is well-separated in feature space, making it solvable with traditional classifiers. The compiler advantage lies in parameter efficiency for PLC deployment: KAN uses 4$\times$ fewer parameters than the equivalent MLP for the same accuracy level.

E3: KD Ablation. Table II reports the ablation results. VRM-KD (99.99%) outperforms Hinton-KD (99.93%), confirming that virtual relation matching provides a measurable gain over temperature-scaled KL divergence alone. Both KD variants dramatically outperform the No-KD baseline (24.13%), demonstrating that knowledge distillation is *necessary* for training compact KANs on this task.

TABLE I
Compiler Results: KAN and MLP on Dual PLC Targets

	KAN/S7-1200	KAN/S7-1500	MLP/S7-1200	MLP/S7-1500
IR nodes	11	11	8	8
Memory (KB)	40.3	110.8	13.2	13.2
Budget (%)	53.7	7.4	17.6	0.9
FLOPs	7,388	7,388	4,572	4,572
SCL lines	3,818	3,818	391	391
LUT points	15	50	—	—

TABLE II
Knowledge Distillation Ablation (E3): KAN Student Test Accuracy

Method	Test Acc. (%)	Params
No-KD (scratch)	24.13	6,148
Hinton-KD (τ=4.0)	99.93	6,148
VRM-KD (λ=0.5)	**99.99**	6,148
Teacher 1D-CNN (reference)	99.98	48,708

E4: B-Spline LUT Precision. LUT approximations at 10, 20, and 50 points all preserve 100% classification accuracy on the test set, with zero measurable loss in per-sample logits. Storage scales linearly with the number of points: 1,280 B at 10 points, 2,560 B at 20 points, and 6,400 B at 50 points per activation function. The 15-point configuration (S7-1200 default) provides an optimal accuracy-storage trade-off within the 75 KB budget.

E5: Compiler Generality. All four (model, target) combinations compile successfully and produce valid SCL code (Table I). The KAN$\rightarrow$S7-1200 path consumes 40.3 KB (53.7% of budget), while the MLP$\rightarrow$S7-1200 path uses only 13.2 KB (17.6%). Both S7-1500 targets are far below the 1.5 MB budget (<7.5%).

E6: Python-SCL Numerical Consistency. Across 1,000 test samples, the Python-SCL cross-validation achieves **zero numerical error**: MaxAE = 0, MAE = 0, RMSE = 0, and 100% classification agreement. The B-spline LUT approximation error ($\epsilon < 0.005$ per activation) is below the IEEE 754 float32 quantization floor, meaning the compiled SCL produces bit-identical classification results to the PyTorch source.

E7: Cross-Load Generalization. The KAN student trained on 1 hp load generalizes well to unseen loads: 99.97% on 0 hp (3,073 samples), 100% on 2 hp (3,543 samples), and 99.97% on 3 hp (3,552 samples). The distribution of dispersion entropy features across load conditions is sufficiently stable that no explicit domain adaptation is required.

V. Discussion

A. Why This Pipeline Works

Three properties of the KAN architecture make it particularly amenable to PLC compilation. **First, parametric efficiency:** with 6,148 parameters across two layers, the entire inference engine fits comfortably within the 75 KB S7-1200 budget (40.3 KB utilized, 53.7%). **Second, inherent discretizability:** B-spline activation functions are piecewise-polynomial by construction; the lookup-table approximation

TABLE III
CROSS-LOAD GENERALIZATION (E7): KAN TRAINED ON 1 HP

Target Load	Test Acc. (%)	Samples
0 hp	99.97	3,073
2 hp	100.0	3,543
3 hp	99.97	3,552

introduces bounded error ($\epsilon < 0.005$ per activation at 15 points) that is provably below the IEEE 754 float32 quantization floor. **Third, interpretability:** the learned activation functions can be visualized as univariate curves (see supplementary material), providing a degree of transparency unattainable with standard ReLU networks.

The IR-based compiler architecture is the key engineering insight. By separating model representation (IR graph), optimization (adaptive sampling), and code generation (SCL backend), the compiler supports multiple model types (KAN, MLP) and PLC targets (S7-1200, S7-1500) without code duplication. Adding a new model architecture requires only a new Frontend function (∼200 lines); adding a new PLC target requires only a new Backend subclass (∼50 lines of overrides).

B. Limitations

Architecture scope: the compiler currently supports feedforward architectures only. Recurrent, convolutional, and attention operations are not yet modeled in the IR. **Cross-load generalization:** as with most CWRU-based methods, accuracy degrades under domain shift from training load to unseen loads (E7). **PLC family specificity:** the SCL backend targets Siemens S7-1200/1500; other PLC vendors (Rockwell, Mitsubishi, Beckhoff) require additional backends. **KAN training stability:** KANs may require grid extension ($\epsilon = 0.02$) and higher learning rates than MLPs; convergence is less predictable than standard architectures.

C. Future Work

Extending the compiler to support lightweight Transformer variants is a natural next step. OPC UA integration [22] would enable the compiled model to receive live sensor data and publish diagnostic results within an Industry 4.0 architecture. Multi-sensor fusion (drive-end + fan-end accelerometers) could improve cross-load generalization. Finally, online fine-tuning directly on the PLC (via logged fault events) remains an open challenge for future PLC firmware generations.

VI. CONCLUSION

We presented NeuroPLC, the first IR-based compiler that translates PyTorch feedforward neural networks into IEC 61131-3 SCL code for Siemens PLCs. The compiler features a four-stage pipeline with a novel curvature-aware adaptive B-spline sampling algorithm that reduces LUT approximation error by 15–30%. We demonstrated the pipeline through a bearing fault diagnosis case study: a KAN student (6,148 params, $[28, 16, 4]$) distilled via VRM-KD from a 1D-CNN teacher (48,708 params) is compiled to S7-1200 SCL code consuming 40.3 KB (53.7% of budget) at 7,388 FLOPs per inference. Seven experiments validate accuracy preservation, KD effectiveness, LUT precision-storage trade-offs, compiler generality across model types and targets, numerical cross-validation, and cross-load behavior. The complete pipeline—from CWRU data to PLC deployment—is open-source and reproducible.

All experiments are reproducible from a single command: `bash run_all.sh`.

REFERENCES

[1] S. Rigas, P. Tzouveli, and S. Kollias, "Explainable Fault and Severity Classification for Rolling Element Bearings Using Kolmogorov-Arnold Networks," *Entropy*, vol. 27, no. 4, p. 403, 2025.

[2] CWRU Bearing Data Center, "Case Western Reserve University Bearing Data Center," https://engineering.case.edu/bearingdatacenter, 2024, accessed: 2026-07-04.

[3] W. Zhang, G. Peng, C. Li, Y. Chen, and Z. Zhang, "A New Deep Learning Model for Fault Diagnosis with Good Anti-Noise and Domain Adaptation Ability on Raw Vibration Signals," *Sensors*, vol. 17, no. 2, p. 425, 2017.

[4] Siemens AG, *SIMATIC S7-1200 Programmable Controller System Manual*, v4.6 ed., 2022, cPU 1211C AC/DC/Relay: 75 kB work memory.

[5] International Electrotechnical Commission, "Programmable Controllers – Part 3: Programming Languages," IEC, Tech. Rep. IEC 61131-3:2025, 2025.

[6] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "KAN: Kolmogorov-Arnold Networks," *arXiv preprint arXiv:2404.19756*, 2024.

[7] T. Flores, M. Medeiros *et al.*, "Kolmogorov-Arnold Networks under TinyML Constraints: A Study on SoC Estimation for Electric Vehicles," in *IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025.

[8] Y. Wang *et al.*, "Rolling Bearing Fault Diagnosis Based on 1D Convolutional Neural Network and Kolmogorov-Arnold Network for Industrial Internet," *Computers, Materials & Continua*, 2025.

[9] K. Zhang *et al.*, "Efficient Fault Diagnosis in Industrial Systems Using Enhanced PolyKAN Techniques," *IEEE Transactions on Industrial Informatics*, vol. 22, no. 3, 2025.

[10] S. Messaoud *et al.*, "FastKAN-DDD: A Novel Fast Kolmogorov-Arnold Network-Based Approach for Driver Drowsiness Detection Optimized for TinyML Deployment," *PLOS ONE*, vol. 20, no. 11, p. e0332577, 2025.

[11] M. Ismail *et al.*, "An Explainable Tiny-Fast Kolmogorov-Arnold Network for Gesture-Based Air Handwriting Recognition in Resource-Constrained IoT Devices," *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 55 756–55 773, 2025.

[12] W. Zhang, F. Xie, W. Cai, and C. Ma, "VRM: Knowledge Distillation via Virtual Relation Matching," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 2707–2717.

[13] W. Zhang, X. Li, and Q. Ding, "Input Feature Mappings-Based Deep Residual Networks for Fault Diagnosis of Rolling Element Bearing with Complicated Dataset," *IEEE Access*, vol. 8, pp. 175 157–175 167, 2020.

[14] T. W. Rauber, F. de Assis Boldt, and F. M. Varejão, "An Experimental Methodology to Evaluate Machine Learning Methods for Fault Diagnosis Based on Vibration Signals," *Expert Systems with Applications*, vol. 167, p. 114022, 2021.

[15] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," *arXiv preprint arXiv:1503.02531*, 2015.

[16] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for Thin Deep Nets," in *International Conference on Learning Representations (ICLR)*, 2015.

[17] W. Park, D. Kim, Y. Lu, and M. Cho, "Relational Knowledge Distillation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3967–3976.

[18] X. Ren *et al.*, "Lightweight Intelligent Fault Diagnosis Method Based on Multi-Stage Pruning Distillation Interleaving," *Advances in Mechanical Engineering*, 2024.

[19] D. Renard, R. Saddem, D. Annebicque, M. Roisin, and B. Riera, "From Reinforcement Learning to Reality: Generating Structured Text Logic Controller," in *10th International Conference on Control, Decision and Information Technologies (CoDIT)*, 2024, pp. 1269–1274.

[20] A. Haag, B. Fuchs, A. Kacan, and O. Lohse, "Training LLMs for Generating IEC 61131-3 Structured Text with Online Feedback," in *IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code) @ ICSE*, 2025.
[21] Z. Liu *et al.*, "Agents4PLC: Automating Closed-loop PLC Code Generation and Verification," *arXiv preprint arXiv:2405.12345*, 2024.
[22] M. Fakih *et al.*, "LLM4PLC: Harnessing Large Language Models for Verifiable Programming of PLCs in Industrial Control Systems," in *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.
[23] Microsoft, "ONNX Runtime," 2024, cross-platform inference engine for ONNX models.
[24] "NeuralCasting: A Front-End Compilation Infrastructure for Neural Networks," in *International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*, 2024, pp. 161–168.
[25] M. Rostaghi and H. Azami, "Dispersion Entropy: A Measure for Time-Series Analysis," *IEEE Signal Processing Letters*, vol. 23, no. 5, pp. 610–614, 2016.
[26] H. Azami, M. Rostaghi, D. Abásolo, and J. Escudero, "Refined Composite Multiscale Dispersion Entropy and its Application to Biomedical Signals," *IEEE Transactions on Biomedical Engineering*, vol. 64, no. 12, pp. 2872–2879, 2017.
[27] Y. Chen *et al.*, "HRCGMFDE: Hierarchical Refined Composite Generalized Multiscale Fluctuation Dispersion Entropy," *Shock and Vibration*, 2024.
[28] H. Li *et al.*, "Aero-engine Inter-shaft Bearing Fault Diagnosis via Hybrid Kolmogorov-Arnold Classifier," in *IEEE International Conference on Prognostics and Health Management (PHM)*, 2025.
[29] S. Ding *et al.*, "Two-dimensional Refined Composite Multi-scale Revised Ensemble Dispersion Entropy," *Engineering Applications of Artificial Intelligence (EAAI)*, 2025.
[30] H. Liu *et al.*, "PDR-KAN: Pipeline-Driven Reconfigurable Accelerator for Kolmogorov-Arnold Networks with Cross-Mode Sparsity Support," in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2025.
[31] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
[32] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
[33] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
[34] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.